DVP WEEK 14 ASSIGNMENT

EC2 Monitoring

with Prometheus + Grafana

SUBMITTED BY	Shijo Shalom S
PROJECT	EC2 Monitoring using Prometheus and Grafana
PLATFORM	AWS EC2 / Ubuntu
SUBMISSION DATE	12 October 2026

TECHNOLOGY

Prometheus • Node Exporter • Grafana • Gmail SMTP • AWS EC2

Objective

The objective of this project is to monitor an AWS EC2 Ubuntu instance using Prometheus and Grafana. The system collects host metrics through Node Exporter, displays CPU, memory, disk, and network usage in a dashboard, and sends email notifications when CPU utilization remains above 80% for five minutes.

Technologies and Tools Used

Technology/Tool	Purpose	Usage in Project
AWS EC2 / Ubuntu	Cloud compute	Hosts monitoring services
Node Exporter	Host metrics exporter	Exposes Linux system metrics
Prometheus	Metrics collection and query	Scrapes and stores metrics; PromQL
Grafana	Visualization and alerting	Dashboard, rule, notification routing
Gmail SMTP	Email notification	Firing and resolved alert emails

Monitoring Workflow

- Node Exporter exposes Linux system metrics from the EC2 instance.
- Prometheus scrapes metrics from Node Exporter and stores time-series data.
- Grafana uses Prometheus as a data source to build dashboards and evaluate alert rules.
- When CPU usage stays above the configured threshold for five minutes, Grafana sends an email notification.
- When CPU usage returns to normal, Grafana sends a resolved notification.

Monitoring Concepts

Why is Monitoring essential in DevOps

Monitoring is an essential part of DevOps because it helps teams continuously track the health, performance, availability, and reliability of applications and infrastructure in a production environment.

Monitoring collects data from servers, applications, databases, networks, and other components. It helps DevOps engineers identify problems early, troubleshoot failures, and maintain smooth service delivery.

Key benefits of monitoring

- ✓ Early problem detection: Identifies errors and performance issues before they become major failures.
- ✓ Improved availability: Helps keep applications and services running with minimal downtime.
- ✓ Performance optimization: Identifies slow applications, high CPU usage, and memory problems.
- ✓ Faster troubleshooting: Provides logs, metrics, and alerts to help engineers find the root cause of issues.
- ✓ Better user experience: Ensures applications respond quickly and remain accessible to users.

Five key metrics to monitor in a production system

The following metrics help DevOps engineers understand whether a production system is healthy and performing efficiently.

Metric	What it measures	Why it is important
CPU Utilization	Percentage of CPU resources being used.	Helps identify high processor usage and overloaded servers.
Memory Utilization	Amount of RAM currently being used.	Helps detect memory shortages and potential application failures.
Disk Usage	Storage space used by the system.	Prevents disk exhaustion that could

		affect applications and services.
Response Time	Time taken by an application to respond to a request.	Helps identify slow applications and improve user experience.
Error Rate	Percentage or number of failed application requests.	Helps detect software bugs, service failures, and other issues.

Comparison of three popular monitoring tools



Prometheus

Type: Open-source monitoring and alerting

Prometheus collects and stores time-series metrics, supports queries using PromQL, and integrates well with Kubernetes and Grafana.

Pros

- Free and open-source.
- Excellent for cloud-native and Kubernetes environments.
- Supports flexible queries and alerting.

Cons

- Requires setup and maintenance.
- Long-term storage and large-scale deployments may need additional components.
- Primarily focused on metrics rather than full log management.



Datadog

Type: Cloud-based observability platform

Datadog provides infrastructure monitoring, application performance monitoring (APM), log management, dashboards, and alerting through a unified platform.

Pros

- Easy-to-use dashboards and visualizations.
- Combines metrics, logs, and traces.
- Integrates with many cloud platforms and applications.

Cons

- Can become expensive as data volume and monitored resources increase.
- Depends on a third-party cloud service.
- Configuration and pricing can be complex for large environments.



Nagios

Type: Infrastructure and network monitoring

Nagios monitors servers, network devices, services, and system resources. It can notify administrators when monitored components fail or cross configured thresholds.

Pros

- Mature and widely used monitoring solution.
- Good for server, network, and service availability checks.
- Nagios Core is open-source and supports plugins.

Cons

- Interface and configuration can feel less modern.
- Advanced configuration may require technical expertise.
- Detailed application tracing and advanced analytics often require additional tools.

Three best practices for effective monitoring

- **Set meaningful alerts and thresholds:** Configure alerts for important issues, such as high CPU usage, increased error rates, and application downtime. Avoid excessive alerts that can cause alert fatigue.
- **Use dashboards and monitor service health:** Create dashboards to visualize key metrics and monitor application availability, latency, errors, and resource utilization.

Track service-level indicators (SLIs) and service-level objectives (SLOs) where appropriate.

- **Review metrics and continuously improve:** Analyze monitoring data regularly, investigate recurring incidents, and adjust alert thresholds as the system changes. Use monitoring results to improve performance, reliability, and capacity planning.

Create a dashboard in Grafana showing EC2 instance

A dashboard named "EC2 Monitoring Dashboard" was created in Grafana using Prometheus as its data source. Node Exporter provides the Linux host metrics. The dashboard contains four required panels.

Panel	What it Shows	Why it is Useful
CPU Usage (%)	Processor utilization	Helps identify sustained CPU pressure.
Memory Usage (%)	RAM used relative to total	Helps detect memory pressure.
Disk Usage (%)	Root filesystem space used	Helps prevent storage exhaustion.
Network Traffic	Received bytes per second	Shows traffic changes over time.

PromQL queries used

- ✓ **CPU usage:** 100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))
- ✓ **Memory usage:** 100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)
- ✓ **Disk usage:** 100 * (1 - node_filesystem_avail_bytes{mountpoint="/",fstype!="rootfs"} / node_filesystem_size_bytes{mountpoint="/",fstype!="rootfs"})
- ✓ **Network receive rate:** rate(node_network_receive_bytes_total{device!="lo"}[5m])

Dashboard best practices applied

The dashboard uses clear panel titles, percentage units for CPU, memory and disk, and a time-series graph for network traffic. Placing related metrics together makes it easier to understand the instance's condition at a glance.

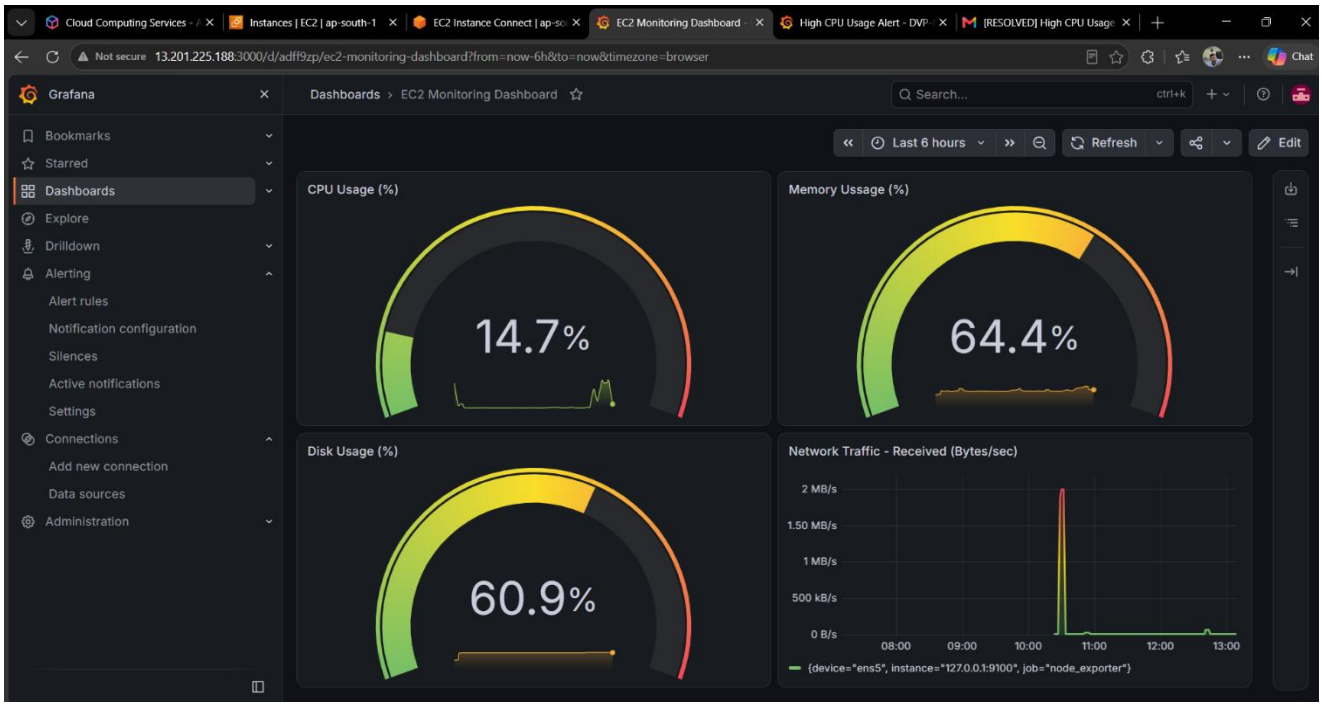


Figure 1. EC2 Monitoring Dashboard displaying the four required metrics.

Alerting in Grafana

A Grafana-managed rule named “High CPU Usage Alert” was configured with Prometheus as its data source. It checks CPU utilization and fires when the result is above 80% for the configured five-minute pending period.

Alert rule	High CPU Usage Alert
Condition	CPU utilization above 80%
Pending period	5 minutes
Evaluation interval	1 minute
Contact point	DVP Email Notifications
Email channel	Gmail SMTP
Verification	Firing and resolved emails received; returned to Normal

Alert query

```
100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))
```

Practical demonstration summary

The monitoring setup uses Prometheus to collect system metrics and Grafana to visualize them and evaluate alert conditions. An alert rule was configured to detect CPU utilization above 80% for five minutes. When the threshold was exceeded, Grafana generated an alert and sent an email notification. After CPU utilization decreased, the alert returned to normal and a resolution email was received.

This practical demonstrates metric collection, dashboard visualization, threshold-based alerting, email notifications, and alert resolution.

Practical implementation and screenshot captions

Prometheus Data Source Configuration

Prometheus was configured as the data source in Grafana to collect and visualize system metrics.

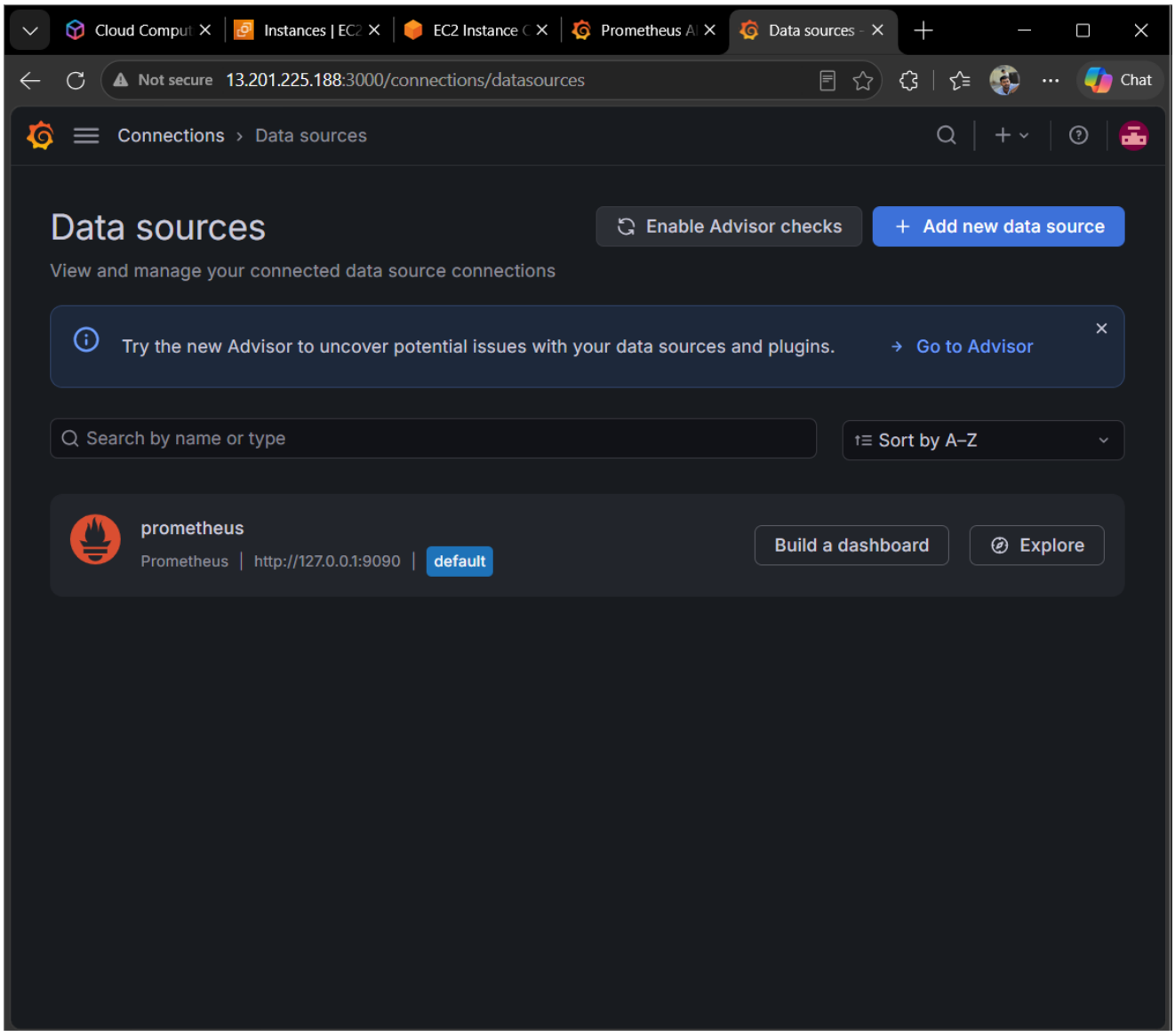


Figure 2. Grafana data sources showing Prometheus configured as the default data source.

Email Notification Configuration

An email contact point and notification policy were configured to receive alerts when monitoring conditions were met.

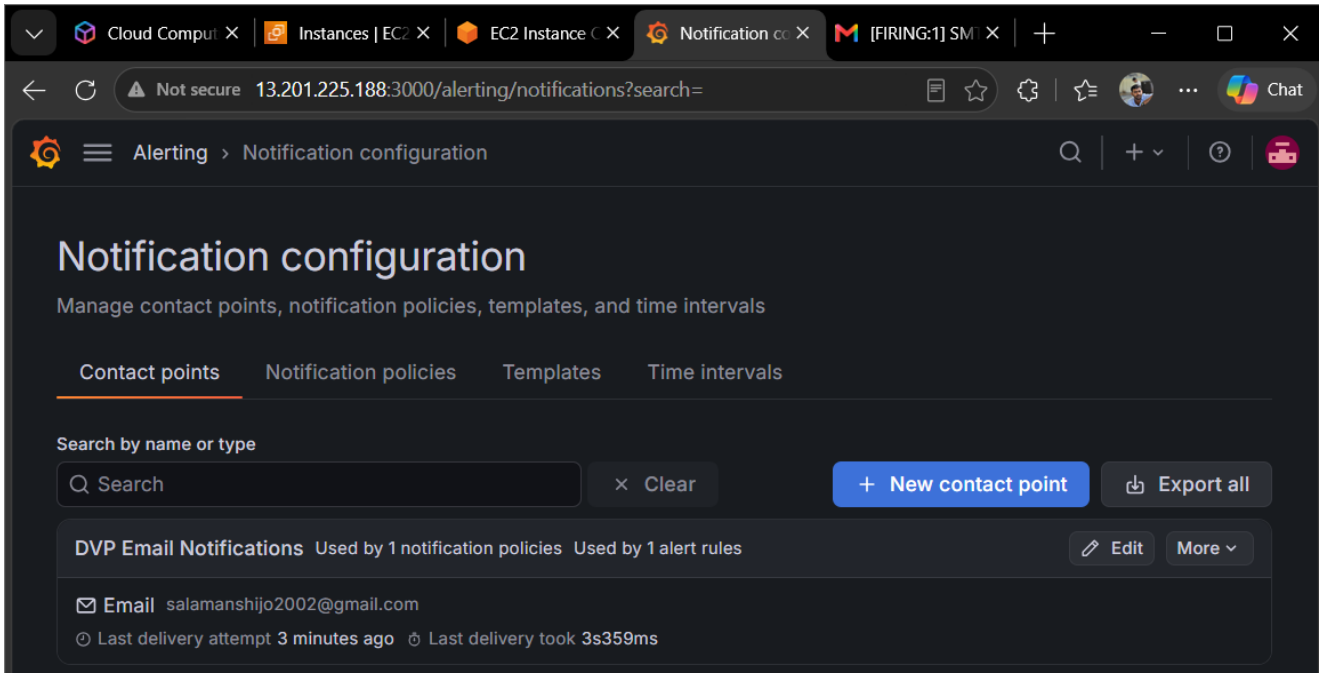


Figure 3. DVP Email Notifications contact point and delivery status.

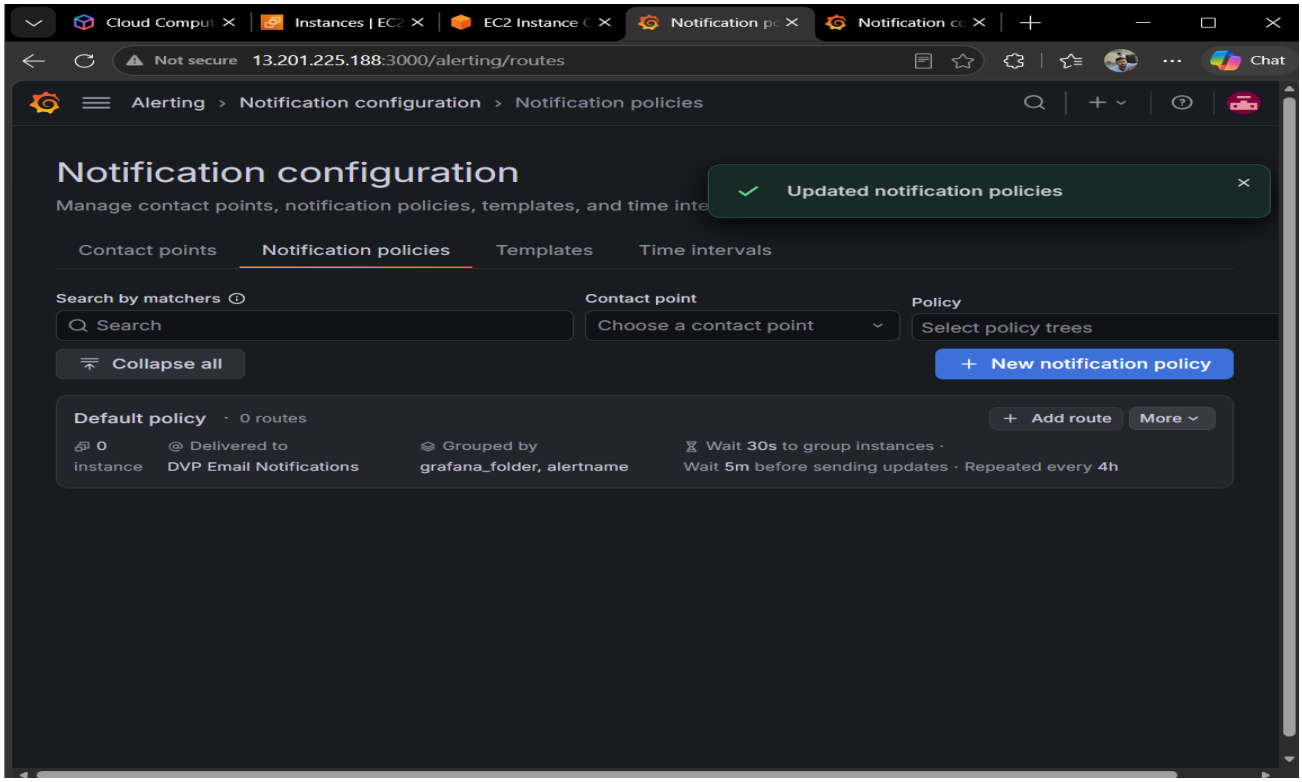


Figure 4. Grafana notification policy routing alerts to DVP Email Notifications.

EC2 Monitoring Dashboard

The Grafana dashboard displays CPU utilization, memory usage, disk usage, and received network traffic for the monitored EC2 instance.

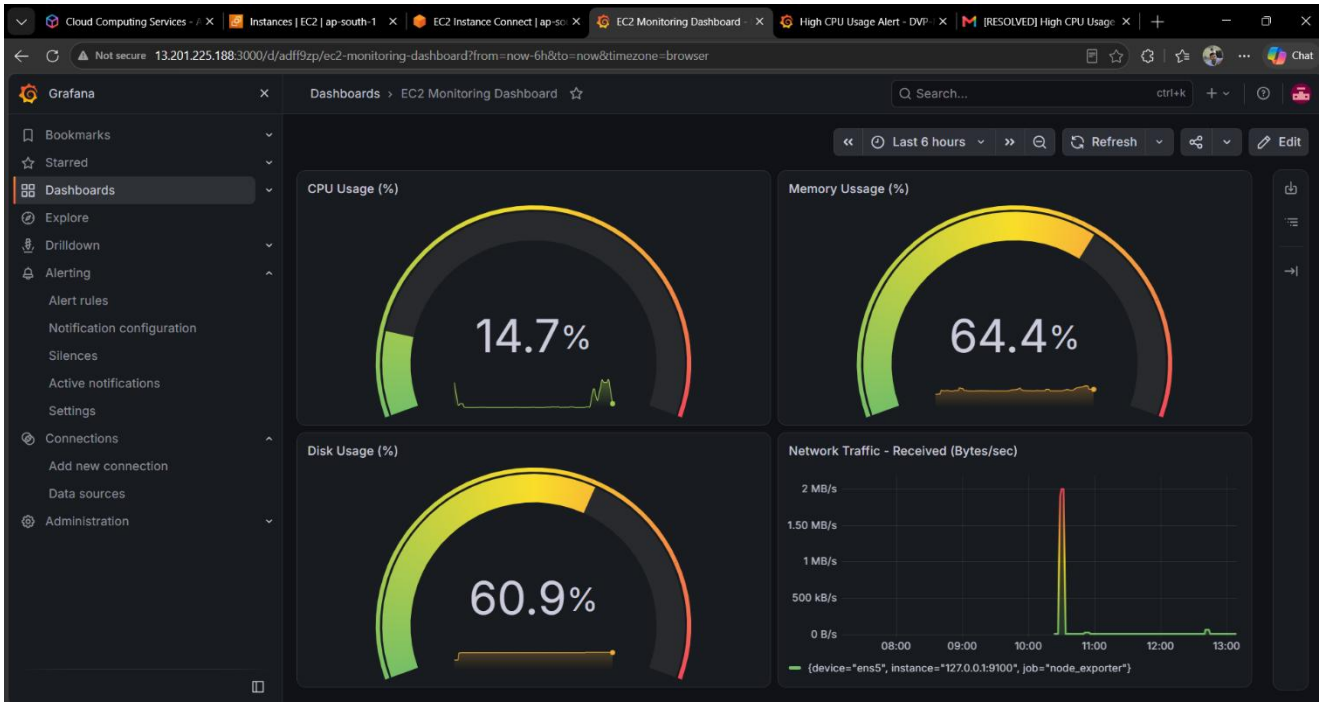


Figure 5. EC2 Monitoring Dashboard

High CPU Usage Alert Rule

A PromQL query calculates CPU utilization, and Grafana evaluates the result against an 80% threshold with a five-minute pending period.

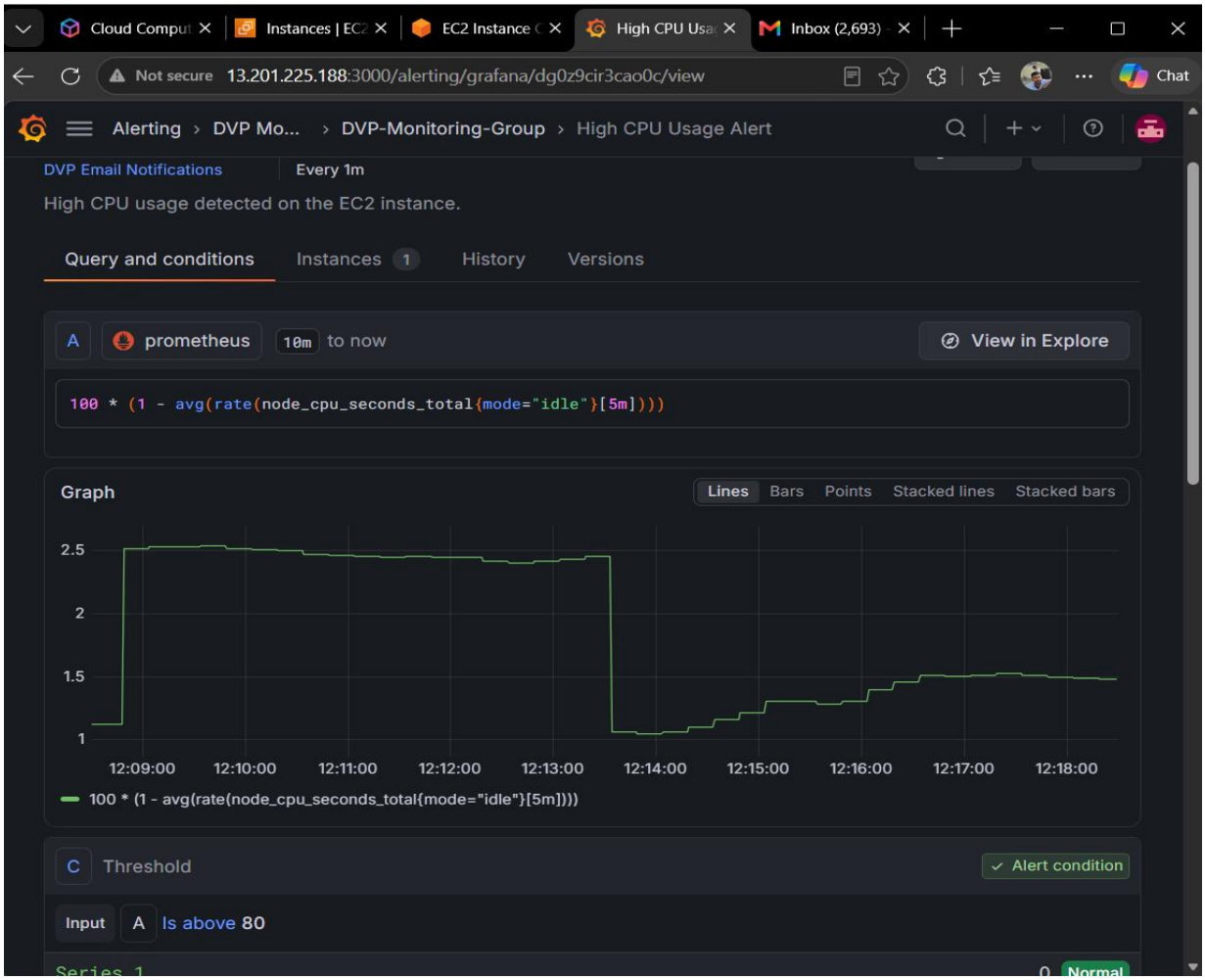


Figure 6. High CPU Usage Alert rule query and threshold configuration.

High CPU Utilization Detected on the EC2 Instance

The Grafana dashboard shows CPU utilization reaching 92.9%, indicating high resource usage. This observation demonstrates the importance of continuous monitoring to identify CPU spikes and trigger alerts when the configured threshold is exceeded.



Figure 7. High CPU Usage

High CPU Alert Email Received

When CPU utilization remained above the configured threshold, Grafana sent an email notification describing the detected condition.

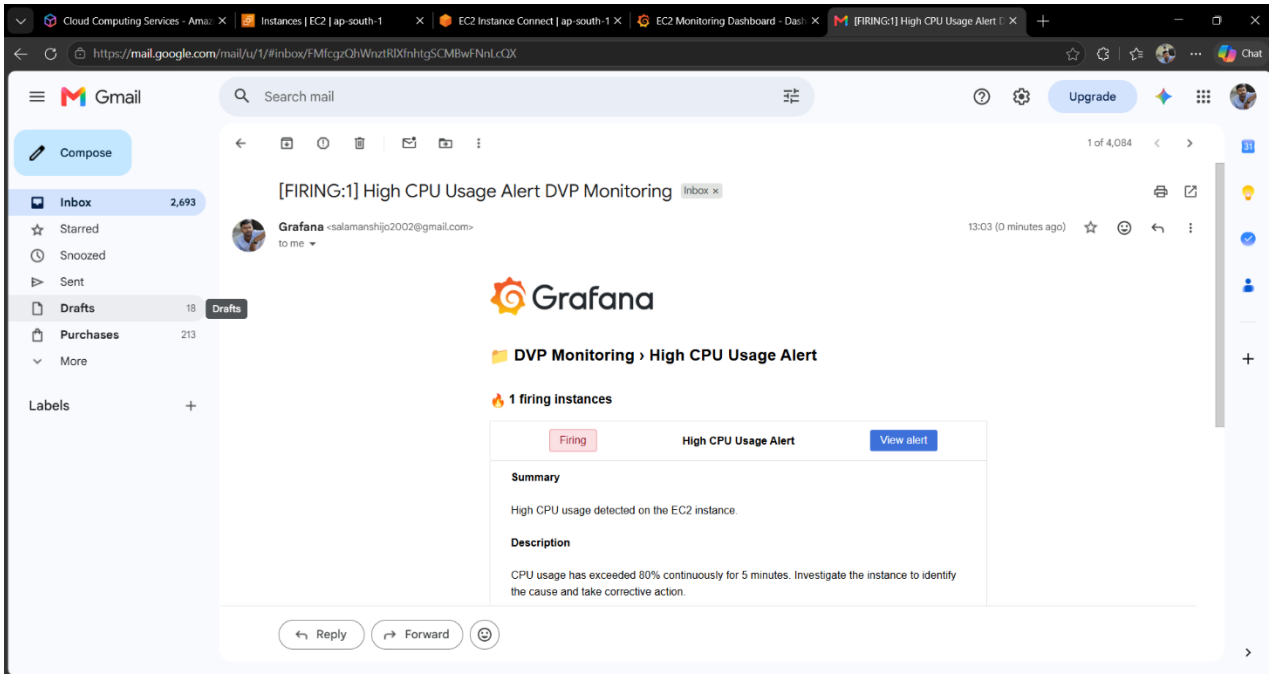


Figure 8. Gmail firing notification for High CPU Usage Alert.

CPU Threshold Breach

The graph shows CPU utilization crossing the 80% threshold, causing the alert to enter the firing state.

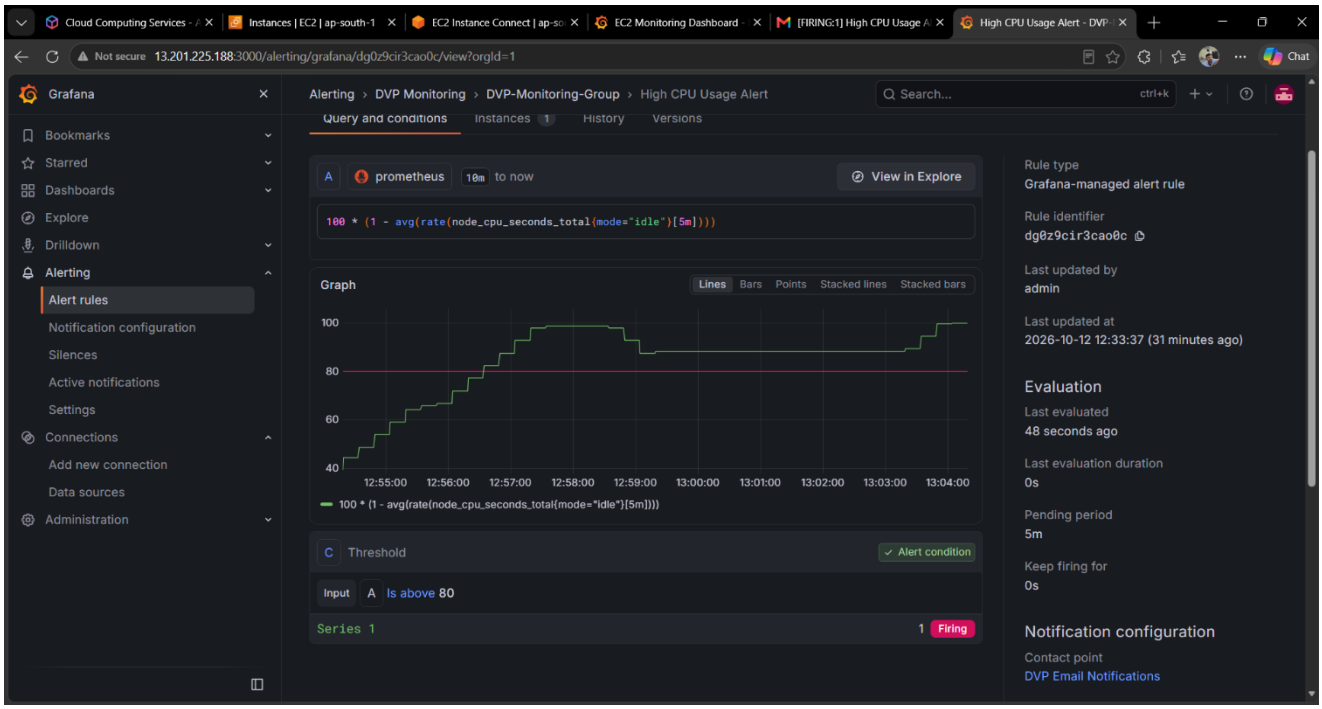


Figure 9. Grafana alert rule showing the 80% threshold and five-minute pending period.

CPU Utilization Returned to Normal Levels

After the high CPU usage event, CPU utilization decreased from 92.9% to 14.7%. The Grafana dashboard confirms that CPU usage returned to a normal level, while memory, disk, and network metrics continued to be monitored.

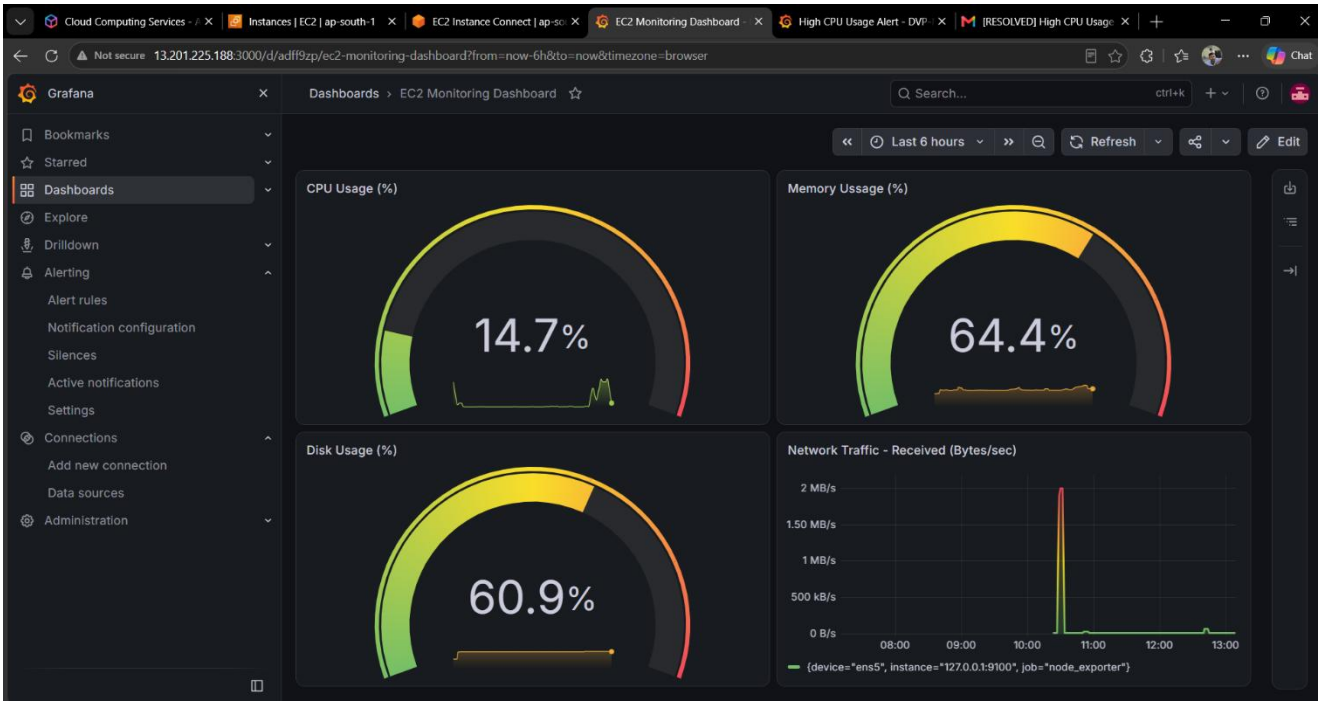


Figure 10. CPU Utilization Returned to Normal State

Resolved Alert Email Notification

Grafana sent a resolution notification after CPU utilization returned below the alert threshold.

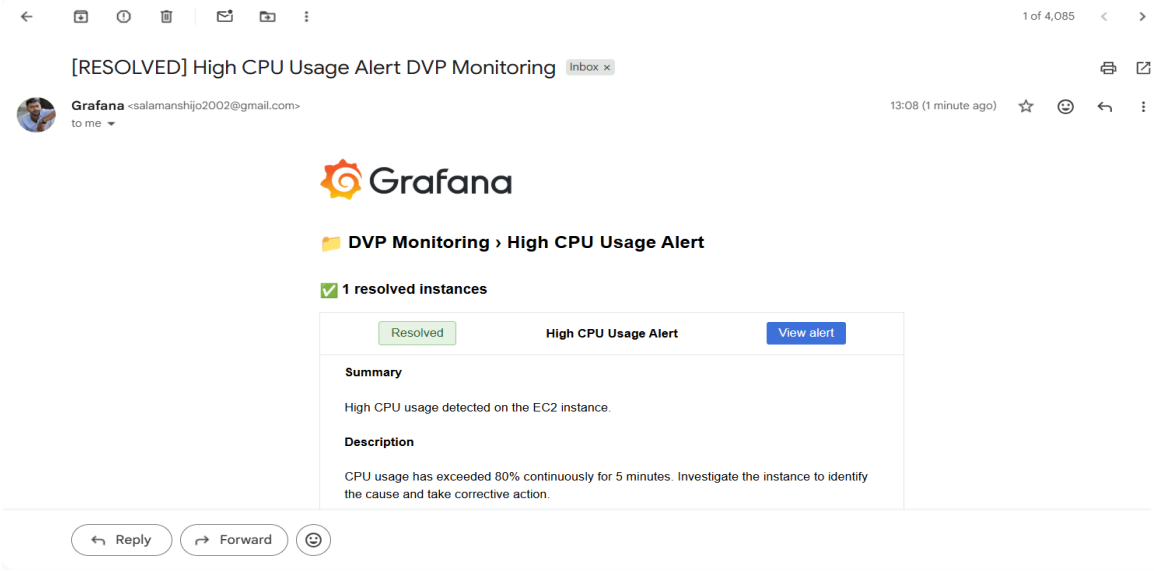


Figure 11. Gmail resolved notification for High CPU Usage Alert.

CPU Usage Alert Resolution in Grafana

Grafana confirms that the high CPU usage alert has returned to the Normal state after CPU utilization dropped below the configured 80% threshold. The graph shows the decrease in CPU usage over time, demonstrating successful alert resolution.

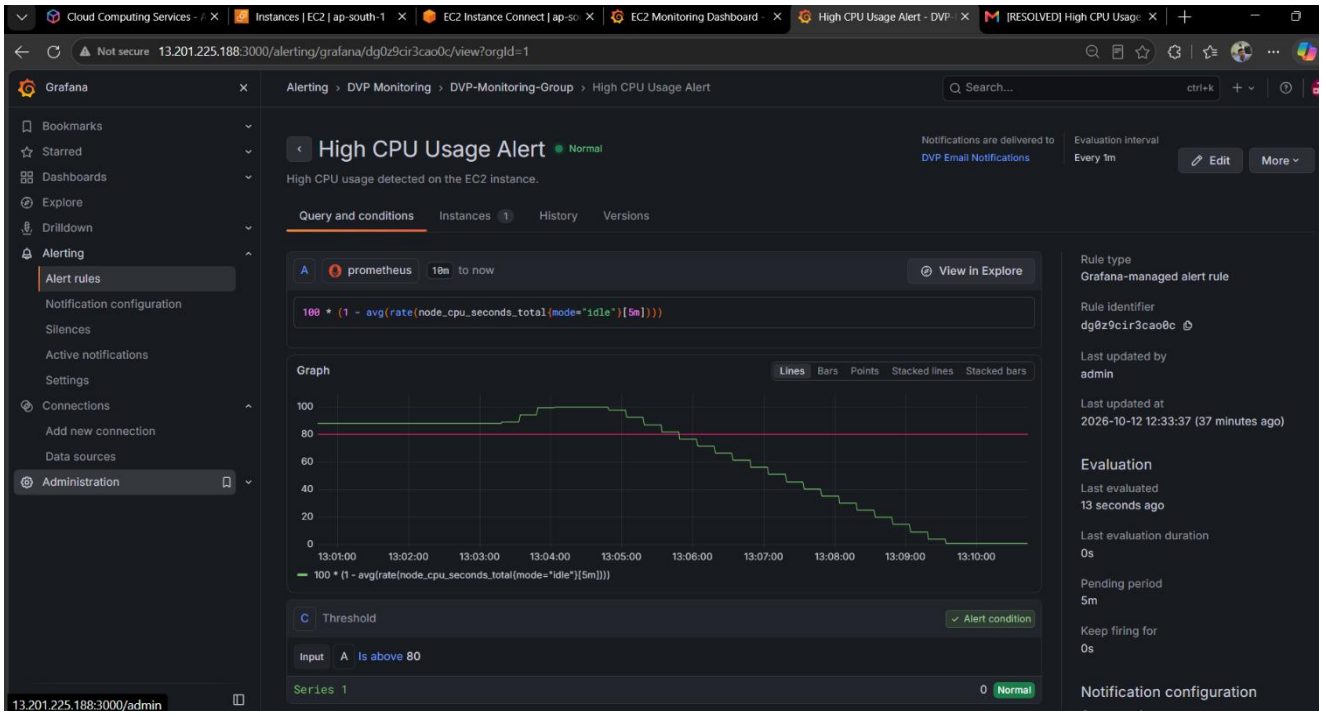


Figure 12. Grafana alert rule in the Normal state after recovery.

Testing result

The CPU stress test raised utilization above the configured threshold and a firing email arrived in Gmail. After the stress test was stopped and CPU utilization returned below the threshold, Grafana showed the alert as Normal and a resolved email was received.

Visualization and Downtime Reduction in DevOps

Importance of Visualization in Monitoring

Visualization is an important part of monitoring because it converts raw system metrics into graphs, charts, and dashboards that are easy to understand. It helps DevOps engineers quickly identify performance issues, unusual resource usage, and potential system failures.

In this project, Grafana was used to visualize CPU utilization, memory usage, disk usage, and network traffic of an Amazon EC2 instance. These visualizations help administrators understand system health, identify performance trends, and make informed decisions. When integrated with Prometheus, Grafana provides a clear view of collected metrics and helps simplify troubleshooting.

Three Best Practices for Creating Grafana Dashboards

- Display relevant metrics clearly: Include important metrics such as CPU utilization, memory usage, disk space, network traffic, response time, and error rate. Use suitable graphs, gauges, and units to make the information easy to understand.
- Use meaningful thresholds and alerts: Set appropriate warning and critical thresholds for important metrics. For example, an alert can be configured when CPU utilization exceeds 80% for five continuous minutes. This helps identify potential problems before they seriously affect the system.
- Keep dashboards simple and organized: Group related metrics into separate panels, use clear titles, and avoid unnecessary visual elements. Configure suitable time ranges and automatic refresh intervals so engineers can quickly understand the current system status.

How Alerts Reduce Downtime in DevOps Environments

Alerts help DevOps teams detect problems quickly and take corrective action before they cause prolonged service interruptions. Instead of continuously checking dashboards manually, engineers receive notifications when predefined conditions are met.

In this project, a Grafana alert rule was configured using Prometheus CPU metrics to detect CPU utilization above 80% for five continuous minutes. When CPU utilization reached 92.9%, the alert entered the *Firing* state, and an email notification was received. After CPU

utilization decreased to 14.7%, the alert returned to the *Normal* state, and a resolution email was received.

This practical demonstration shows how automated alerting helps identify performance issues, reduces the time required to detect and investigate incidents, and supports faster recovery. In real DevOps environments, timely alerts can reduce the impact of application failures, server overload, and resource exhaustion.

Conclusion

This project successfully integrated Prometheus and Grafana to monitor EC2 system metrics and visualize CPU, memory, disk, and network usage. A CPU alert threshold of 80% triggered an email notification when usage reached 92.9%, and a resolution notification was received after usage dropped to 14.7%. This demonstrates how automated monitoring and alerting help detect performance issues early and reduce potential downtime.